

Reliability & Data Layers

Part 4: Middleware, Logging, Guardrails, and RAG

researchcontentlab@gmail.com

Project ID: project-ba58c036-070e-438e-b8b

1. Middleware & Safety

- Located in `api/middleware/`.
- Validates active HTTP headers, decodes JWT user tokens, and isolates sessions.
- **Row-Level Security (RLS)**: Propagates user/session context to PostgreSQL (Supabase) to isolate cart data and histories.

```
@app.middleware("http")
async def db_session_middleware(request: Request, call_next):
    # Retrieve session token
    token = request.headers.get("Authorization", "")
    request.state.session_id = parse_session(token)
    response = await call_next(request)
    return response
```

2. Observability & Logging

- Fully instrumented using **logfire** (Pydantic's observability library) and **OpenTelemetry**.
- Path: `api/observability/`
- Traces the entire execution tree: from HTTP endpoints down to LLM planner completions and SQL queries.
- Connects live logs directly to the developer's dashboard for troubleshooting.

```
import logfire
logfire.configure(pydantic_ai_instrument=True)
logfire.instrument_fastapi(app)
```

3. LLM Guardrails

- Location: `api/guardrails/`
- Standardizes checks on both input and output sequences:
 - **Prompt Injection Detector**: Blocks adversarial commands.
 - **PII Masker**: Redacts user details (passwords, emails, card numbers).
 - **Output Validator**: Enforces schema rules and blocks formatting hallucinations before they reach storefront clients.

4. Semantic Search & RAG

- Location: `api/rag/`
- Leverages **Pinecone vector database** for fast product/FAQ search.
- Pre-embeds catalog descriptions using standard OpenAI text embeddings.

Retrieval Pipeline:

1. Receives natural language user request.
2. Embedding generated from query.
3. Pinecone semantic search yields catalog IDs.
4. Database hydrates complete matching details.
5. Returns rich UI cards back to the user.